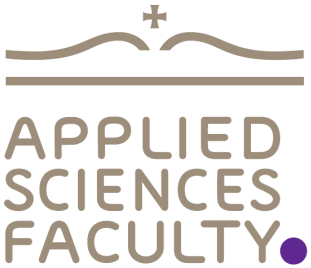
C++ — лекція 21. Огляд STL на прикладі std::vector





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

STL

- STL – Standard Template Library, Стандартна бібліотека шаблонів, складається із трьох основних частин: контейнери, ітератори, алгоритми. Також присутні всілякі допоміжні компоненти — pair, bitset, tuple (C++11) і т.д.
- Розглянемо почнемо із контейнера vector, який можна назвати "контейнером по замовчуванню" — слід використовувати саме його, якщо немає якихось спеціальних вимог, що спонукають скористатися іншими контейнерами.
- Стандартна бібліотека — річ об'ємна (2/3 стандарту присвячено її опису!) Нижче аж ніяк не робитиметься спроба переказати документацію — це просто непрактично. Буде лише загальний огляд можливостей.

vector

- Контейнер vector це шаблон, оголошений в <vector>:

  template< **class** T, **class** Allocator = std::allocator<T> >
  class vector;

  Другий параметр, алокатор[1], займається керуванням пам'яттю, поки нас цілком задовільнятиме алокатор по замовчуванню. Перший параметр — тип елементів, що зберігатимуться в контейнері.
- Вимога до цього типу — для нього повинні існувати конструктор копіювання та оператор присвоєння (C++98, в C++11 вимоги, там де це можливо, послабили).
- **vector** — шаблон класу динамічного масиву. Тобто, може зберігати елементи дуже широкого класу.
- Масив цей може рости та зменшуватися, за потреби — можна додавати та видаляти елементи. Додавання та видалення елементів у кінці відбувається швидко, за амортизований[2] константний час, O(1). Додавання/видалення всередині — O(N).
- Взагалі, STL характеризується тим, що для більшості операції є гарантії щодо алгоритмічної складності.

[1] Див., наприклад, http://ib-krajewski.blogspot.com/2021/12/my-two-talks-about-polymorphic-memory.html.

[2] Тобто, майже завжди за константний час – якщо при цьому вектор не мусить перебудовувати внутрішню структуру, див. далі.

Констуктори

- Створюємо порожні вектори:

```
vector<int> arr1;
vector<double> arr2;
```

- Вектор, розміром 5 елементів із значеннями по замовчуванню, (0.0 в даному випадку), та 7, заповнених числом 1.23:

```
vector<double> arr3(5);
vector<double> arr4(7, 1.23);
```

- Можна також робити копії:

```
vector<double> arr5(arr4);
```

та присвоєння:

```
arr5 = arr4;
```

- Крім того, існує конструктор копіювання, що копіює частину іншого вектора, задану **інтервалом** — про які нижче. C++11 додав конструктори переміщення та конструктор, що отримує список ініціалізації, `initializer_list`, що дозволяє робити так:

```
vector<int> arr6  {1, 2, 3, 4};
vector<int> arr7 = {1, 2, 3, 4}; // Не рекомендована
                                 // форма запису!
```

Робота з елементами — 1

- Звертатися до елементів вектора можна за допомогою звичного оператора `[]`:

```
vector<double> arr3;
arr3[3] = 55;
```

  Вихід за межі вектора при цьому не контролюється, як і для звичайного масиву!

- Для звертання із таким контролем можна скористатися at():

```
cout << arr3.at(55);
```

  У випадку, коли 55-го елемента у векторі немає, буде кинуто виключення `out_of_range`[3].

- Розмір вектора — кількість елементів у ньому, можна взнати за допомогою методу size():

```
for(size_t i = 0; i < arr3.size(); ++i)
    cout << arr3[i] << ", ";
```

  Елементи нумеруються з нуля, від 0 до size()-1, як і для звичайних масивів.

- Метод empty() дозволяє перевірити, чи вектор порожній.

[3] Яке є нащадком `logic_error`, що якби натякає.

Робота з елементами — 2

- Крім довільного доступу за допомогою [] чи at(), можна безпосередньо звернутися до першого і останнього елемента:

```
arr3.front() -= arr3.back();
```

  При чому, так як повертається і посилання, то за їх допомогою можна не тільки читати вміст, але і змінювати його.

- Вектор неперервний у пам'яті, тому, за допомогою `&v[0]` можна отримати звичайний масив C — взявши адресу нульового елемента. В C++11 для цієї ж цілі додали data():

```
void old_print_function(double* ar, size_t sz){
    for(size_t i=0; i < sz; ++i){
        cout << ar[i] << ", ";
    }
}

#if __cplusplus >= 201103L
    old_print_function( arr4.data(), arr4.size() );
#else
    old_print_function( &arr4[0], arr4.size() );
#endif // __cplusplus
```

Модифікація вектора — 1

- Найпростіші способи змінити розмір вектора — додати елемент в кінці:

```
arr1.push_back(5);
arr2.push_back(1.117);
```

  або відкинути останній елемент:

```
arr3.pop_back();
```

  pop_back() не повертає останній елемент — просто викидає його. (Таке розділення — back() для доступу, pop_back() для модифікації, спрощує забезпечення гарантій щодо виключень).
- Повністю очистити вектор можна методом clear():

```
arr1.clear();
assert(arr1.size() == 0);
```

- Змінити розмір можна за допомогою resize():

```
arr2.resize(15);
arr3.resize(22, 3.14);
```

  Зберігає вміст! Якщо новий розмір більший за попередній, по замовчуванню нові елементи заповнюються за допомогою конструктора по замовчуванню, (це не тавтологія!), але за потреби можна вказати необхідне значення.

Модифікація вектора — 2

- Для елементів, що тим чи іншим чином викидаються із вектора, викликається деструктор (але якщо вектор зберігає вказівники, природно, він їх не звільнятиме!)
- Метод **swap** дозволяє обміняти вміст двох векторів місцями:

```
arr3.swap(arr4);
```

Це на додачу до відповідного перевантаження незалежної функції std::swap() для векторів:

```
template <class T, class Alloc>
  void swap (vector<T,Alloc>& x, vector<T,Alloc>& y);
```

- Крім того, існують методи **erase()** — видалити елемент/елементи та **insert** — вставити елементи/елементи. Ці операції, якщо дія відбувається не в кінці, потребують O(N) — відбувається переміщення елементів для того, щоб було заповнити дірку або звільнити місце для нових елементів. Детальніше про них пізніше.
- Потребує додаткової інформації і обговорення методу **assign()**, який дозволяє замінити вміст вектора на вказану кількість копій якогось значення, або скопіювавши частину іншого контейнера. При чому, конструювання відбувається прямо на місці, без виклику оператору присвоєння.

Модифікація вектора — 3

- C++11 додав два зручних методи, `emplace_back()` та `emplace()`, котрі збільшують вектор, а потім конструюють елемент безпосередньо на місці. Вони стали можливими в C++11 завдяки perfect forwarding.

```
struct Drib
{
    int ch, zn;
    Drib(int ich=0, int izn=1): ch(ich), zn(izn){}
};

    vector<Drib> arr8;
    arr8.emplace_back(3, 4);

    arr3.emplace_back(100);
```

- Деякі міркування щодо вибору emplace-функцій: `https://quuxplusone.github.io/blog/2021/03/03/push-back-emplace-back/`.

Оператори відношень

- Для вектора існує весь комплект операцій порівняння та відношень: ==, !=, >, >=, <, <=.
- Оператор рівності, ==, вважає вектори різними, якщо їх розміри різні, або якщо розміри однакові, але трапився відмінний елемент.
- Оператори співвідношення виконуються лексикографічно.
- Увага! Всередині вони користуються лише операторами == та <, відповідно. Тобто, мінімальний комплект операторів, які має підтримувати тип елементів контейнера, щоб користуватися із цими операторами, це є рівність і оператор "менше".
- Для побудови інших співвідношень використовується еквівалентність:

a!=b	!(a==b)
a>b	b<a
a<=b	!(b<a)
a>=b	!(a<b)

capacity — 1

- Щоб надати можливість легко збільшувати розмір, вектор резервує більше пам'яті, ніж йому безпосередньо потрібно. Тоді, поки резерву досить, той же push_back() виконуватиметься за константний час.
- Коли зарезервований об'єм закінчується, вектор збільшує його вдвічі, копіюючи старі дані на нове місце — щоб залишатися неперервним у пам'яті. Тому, власне, стверджується, що операції виду push_back() мають *амортизований* час O(1) — якщо потрібна така реалокація, час може бути лінійним.
- Подивитися, скільки зарезервовано, можна методом capacity(). Не слід плутати його із size()!

```
cout << arr3.capacity() << endl;
```

- Якщо кількість елементів відома наперед, можна зекономити на повторних реалокаціях, зразу зарезервувавши місце:

```
arr3.reserve();
```

Якщо вимагається зарезервувати більше місця, ніж вже зарезервовано, відбувається реалокація, інакше *нічого не відбувається* — reserve() не вміє звільняти виділену вектором пам'ять. Зрозуміло, що reserve() не змінює елементів, чи розміру вектора.

capacity — 2

- В C++98 немає безпосереднього способу зменшити пам'ять, що використовується. Однак, для цього можна скористатися наступною ідіомою:

```
vector<double>( arr ).swap( arr  );
```

Тут створюється анонімна копія вектора arr, яка, однак, буде мати зарезервовано або стільки елементів,скільки реально є в arr, або трішки більше (в залежності від реалізації стандартної бібліотеки), після чого ця копія обмінюється із оригіналом, і деструктор копії звільняє зарезервовану раніше "оригіналом" arr пам'ять. Більш багатослівно це можна записати так:

```
{
    vector<double> tmp( arr )
    tmp.swap( arr  );
} // <--- тут знищується тимчасова змінна tmp
```

- Так само можна очистити вектор, із звільненням пам'яті (clear() її не звільняє), обмінявши із порожнім вектором:

```
vector<double>( ).swap( arr );
```

capacity — 3

- В C++11 з'явився спеціалізований метод, shrink_to_fit():

```
arr.shrink_to_fit();
```

  Скільки пам'яті реально залишиться зарезервованою, знову ж таки, на совісті системи. Зокрема, може приводити до реалокації, але не обов'язково робить її.

Зауваження

- reserve() і capacity(), фактично, деталі реалізації, доступ до яких надано для оптимізації!

Вектор надає все необхідне, щоб реалізувати такий важливий контейнер, як стек:

```
template <class T>
class SimpleStack{
private:
        vector<T> data;
public:
    SimpleStack(){};
    void push(const T& in){ data.push_back(in); }
    bool empty() const     { return data.empty(); }
    size_t size() const    { return data.size(); }

    T pop(){
        T res=data.back();
        data.pop_back();
        return res;
    }
};
```

Зауважте, що тут можна було використати як композицію, так і приватне успадкування. Але успадкування — більш тісний зв'язок, його можна використовувати лише коли воно необхідне, наприклад, є потреба перевизначати віртуальні функції.

Адаптер std::stack

- Корисність такого класу для багатьох користувачів очевидна, тому стандартна бібліотека надає схожу його реалізацію, так званий *адаптер* — клас, що обмежує частину можливостей того чи іншого контейнера, надаючи звичний інтерфейс, скажімо — стека, в нашому випадку:

```
template <class T, class Container = deque<T> >
class stack;
```

- По замовчуванню стек використовує ще один контейнер, deque, але замість нього можна застосувати і vector:

```
stack<double> s2;
stack<double, vector<double> > s3;
```

std::stack надає обмежену (відносно vector) кількість методів:

- Конструктор, який може приймати вже заповнений контейнер, того типу, що й використовуватиметься стеком:

```
explicit stack (const container_type& ctnr = container_type());
```

При тому, вміст контейнера копіюється у внутрішнє представлення.

- Наступні методи, із природною семантикою:

```
void push (const value_type& val);
bool empty() const;
size_type size() const;
void pop();
     value_type& top();
const value_type& top() const;
```

Також наявні оператори відношень, ==, !=, >, >=, <, <=. Порівняння відбувається лексикографічно.

- C++11 додав метод та незалежну функцію swap() та метод emplace():

```
template <class... Args> void emplace (Args&&... args);
```

vector<bool> — 1

- Згідно стандарту vector<bool> є не звичайним контейнером. Це спеціалізація, яка зберігає булівські значення в окремих бітах.
- Ідея цієї спеціалізації була простою — показати приклад роботи із проксі-контейнерами, контейнерами, робота із елементами яких не може відбуватися безпосередньо. Замість посилання на елемент, проксі-контейнер надає проксі-об'єкт, який і забезпечує безпосередній доступ.
- Однак, виявилося, що неможливо створити проксі-контейнер повністю сумісний із вимогами STL до контейнерів. Тому vector<bool>, фактично, не є контейнером (хоча і є "майже контейнером"). Зокрема, він не містить в собі bool, тому не може повернути посилання на нього, bool&, повертаючи проксі-посилання, що суперечить вимогам STL.
- Ще одним мінусом є те, що даний підхід – економія пам'яті (зберігаючи bool в окремих бітах), за рахунок меншої швидкодії (маніпуляція окремими бітами повільніша за роботу із bool), нав'язується всім користувачам.

vector<bool> — 2

- Приклад. Наступний, популярний в певних задачах, код, не скомпілюється із vector<bool>:

```
vector<bool> v; .....
bool* p1 = &v[0];
bool* p2 = &v.front();
```

- Нічого поганого в проксі-контейнерах немає! Просто їх неможливо зробити повністю сумісними із контейнерами STL.
- Щоб отримати повноцінний контейнер bool, можна створити vector<char>, інтерпретуючи їх як булівські значення, або скористатися deque<bool>. Другий спосіб більш коректний, однак, якщо потрібна неперервність в пам'яті, яку забезпечує тільки vector — доводиться користуватися першим, хоч він і покладається на приведення типів.
- Детальніше див. Герб Саттер. "Решение сложных задач на C++", задача 1.13, "Контейнеры, указатели и неконтейнеры".
- ◇ Також: "On vector<bool>".

Ітератори

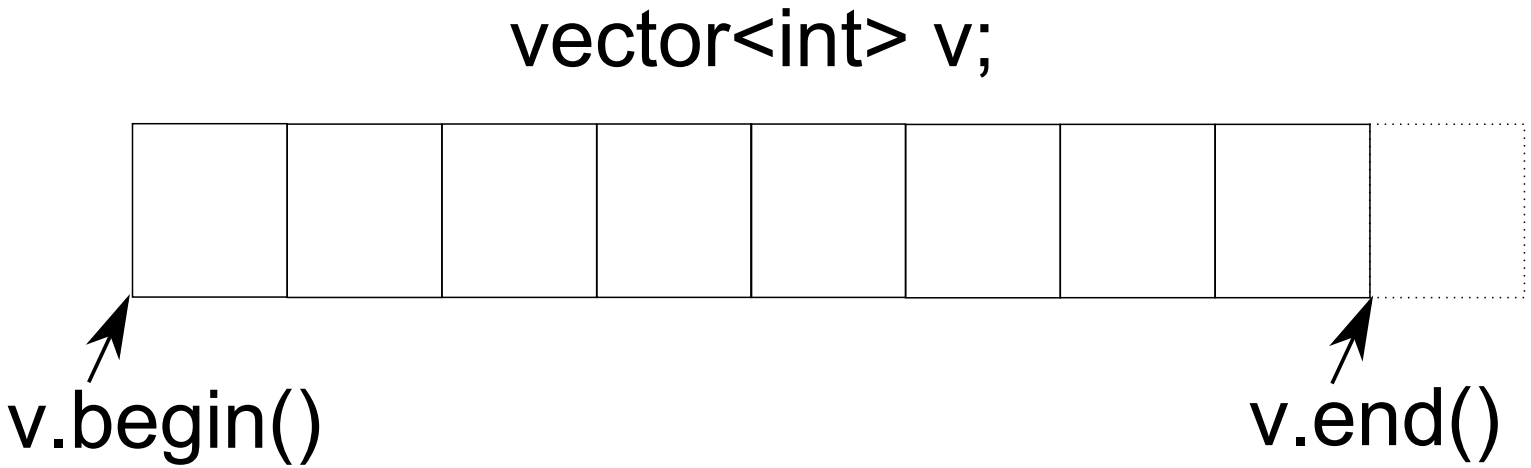
- Для роботи із вектором можна використовувати індекси, або й, враховуючи його неперервність — вказівники (хоча це й не дуже хороша ідея). Однак, це можливе лише завдяки обом умовам: вектор надає доступ за індексами, і неперервний в пам'яті.
- Багато структур даних, наприклад — зв'язні списки, не надають індексів (за самою своєю природою — відсутності ефективного і економного способу створити індекс), і не придатні для використання арифметики вказівників.
- Для організації зручної та загальної (одноманітної) роботи із різноманітними контейнерами, в STL було включено **ітератори** — фактично, узагальнені вказівники. Тобто, вказівники (з нюансами) можна вважати частковим випадком ітераторів.
- Класи-ітератори перевантажують стандартні операції вказівників — розіменування, звертання до члена за вказівником, арифметичні операції (там, де вони мають сенс), порівняння.
- Детальніше вони розглядатимуться в подальшому, зараз розглянемо їх з точки зору роботи із векторами.

- Тип ітератора для контейнера оголошено в ньому самому, тому створити ітератор можна так:

```
vector<int>::iterator itr;
```

- Отримати ітератор на початок контейнера можна за допомогою методу begin(), на (відсутній!) елемент, після останнього[4] — за допомогою методу end().
- Оскільки вектор є структурою довільного доступу, як і масив, для ітераторів реалізована звична арифметика вказівників.
- Паралель із вбудованими масивами (N – ціла константа):

Контейнер:	`int arr[N];`	`vector<int> varr;`
Оголошення ітератора:	`int* ptr;`	`vector<int>::iterator itr;`
Ітератор на початок:	`ptr = arr;`	`itr = varr.begin();`
Ітератор на кінець[5]:	`ptr = arr + N;`	`itr = varr.end();`
Звертання до елемента:	`cout << *ptr;`	`cout << *itr;`
Переміщення до наступного:	`++ptr;`	`++itr;`
та попереднього елемента:	`--ptr;`	`--itr;`
Довільні переміщення:	`ptr += n;`	`itr += n;`
		`std::advance(itr, n);`
Віддаль між ітераторами:	`ptr2 - ptr1;`	`itr2 - itr1;`



- Наприклад, обійти всі елементи контейнера можна так:

```
vector<int>:: iterator   itr ;
for( itr  = v.begin();  itr !=v.end();  ++itr)
{
     cout  << *itr << endl;
}
```

Константні ітератори

- Для константних контейнерів можна скористатися константним ітератором:

```
vector<int>::const_iterator itr;
for(itr = v.begin(); itr!=v.end(); ++itr)
{
    cout << *itr << endl;
}
```

- Вони є аналогами вказівника на константні дані:

const T* ptr <----------> const_iterator

- Тип const iterator відповідає вказівнику, який не можна змінювати:

T* **const** ptr <----------> **const** iterator

Нарешті:

const T* **const** ptr <----------> **const** const_iterator

- Тобто, перетворення між const_iterator і iterator за допомогою const_cast здійснити не вдасться — це, в загальному випадку, різні класи.

Підтримка константних ітераторів

- До C++11 використання константних ітераторів було сильно утрудненим.
  - Детальніше про цю проблему із const_iterator в C++98/03 див. книгу Скотта Мейєрса "Эффективное использование STL" (Scott Meyers, Effective STL), порада номер 26.
- Ця підтримка включає:
  - Багато які алгоритми тепер приймають const_iterator.
  - З'явилися методи cbegin(), cend() і т.д., зокрема, для взаємодії із auto.
- Тому зараз, враховуючи важливість константності, варто користуватися const_iterator за кожної нагоди — як і звичайним const.

Ітератори та auto — C++11

- Писати оці простирадла: `vector<int>::const_iterator itr` незручно. А уявіть собі ітератор на контейнер ітераторів:

  map< vector<**int**>::const_iterator, list <**double**>::iterator >:: const_itera

  (map і list — теж контейнери, про них пізніше).
- В C++98 проблема частково вирішувалася typedef:

  typedef vector<**int**>::const_iterator citr_t;

  Воно позбавляло від необхідності перенабирати згадані простирадла, але активно розмножувало сутності — імена типів різних ітераторів, (звалюючись у своєрідну угорську нотацію, в результаті). З іншого боку, конкретний тип нас, насправді, не цікавить!
- Використання auto із C++11 цю проблему вирішує практично повністю:

```
for(auto itr = v.begin(); itr !=v.end(); ++itr){
    cout << *itr << endl;
}
```

Константні ітератори та auto — C++11

- auto нормально взаємодіє із const, однак const_iterator — інша сутність, не просто "const iterator" (та ж сама різниця, що між константним вказівником і вказівником на константні дані) тому, для простоти, контейнери C++11 також мають методи cbegin(), cend(), які в явному вигляді повертають значення типу const_iterator.

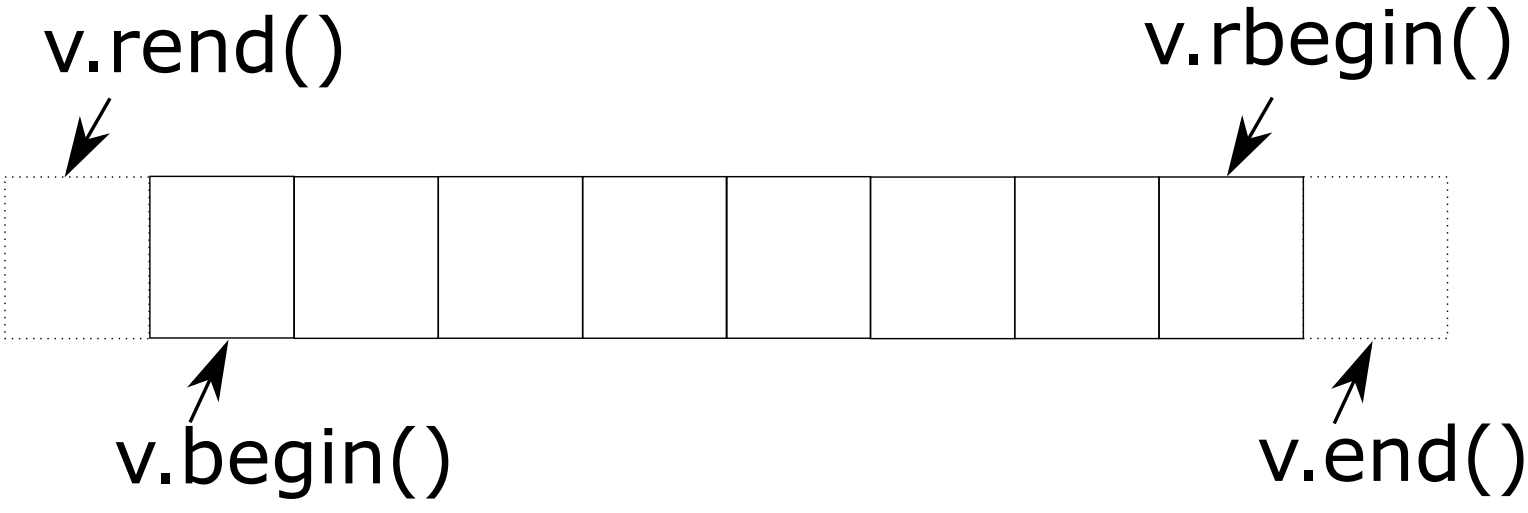
```
template <class T>
void print_vector( const vector<T> & v)
{
    for( auto itr = v.cbegin(); itr !=v.cend(); ++itr)
    {
        cout << *itr << endl;
    }
}
```

Reverse iterators

- Крім звичайних ітераторів, що рухаються вперед, існують також ітератори для обходу контейнерів у зворотному порядку — reverse-ітератори:

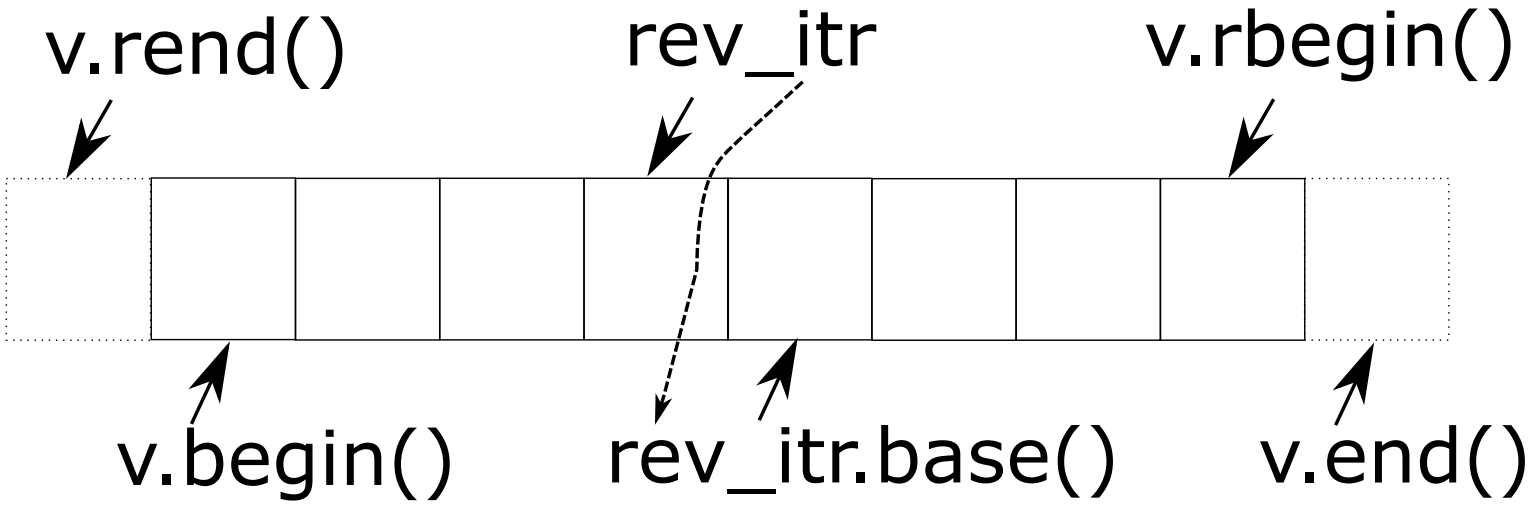
```
vector<int>:: reverse_iterator ritr ;
for( ritr = v.rbegin(); ritr !=v.rend(); ++ritr)
{
    cout << *ritr << endl;
}
```

- Для доступу до них є відповідні методи, rbegin(), rend():



- Існує і його константний варіант: `const_reverse_iterator`.

Нюанси reverse-ітераторів — 1



- Багато методів очікують звичайних ітераторів, iterator, а аж ніяк не reverse-ітераторів. Метод base() повертає ітератор, *еквівалентний* вихідному reverse-ітератору. Однак, з цією еквівалентністю не все так просто.
- Нехай маємо вектор, заповнений 1, 2, 3, 4, 5 і наступний код:

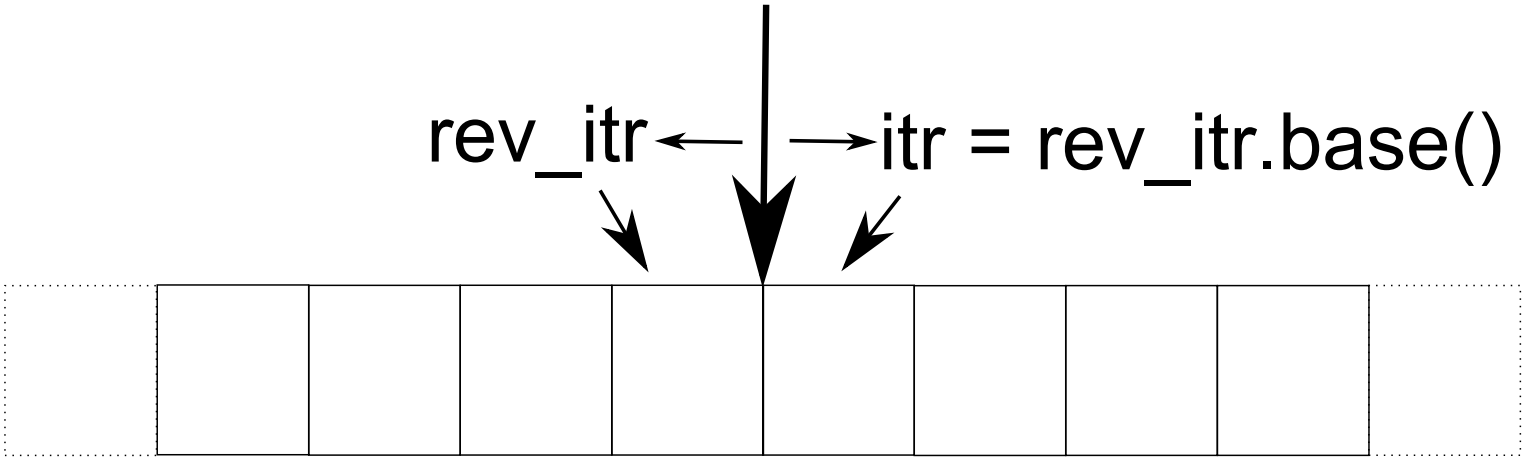
```
ritr = v.rbegin() + 2;
cout << "Reverse iterator: " << *ritr << endl;
cout << "Reverse iterator base: " << *(ritr.base()) << endl;
```

Виведеться:

```
Reverse iterator: 2
Reverse iterator base: 3
```

Нюанси reverse-ітераторів — 2

- Тобто, еквівалентний ітератор вказує на наступний елемент, відносно того, на який вказує reverse-ітератор.
- Скот Меєрс у своїй книзі "Эффективное использование STL" (порада 28) наголошує, reverse-ітератори — плутані по своїй природі, однак, хоча певні труднощі з ними є, якщо уявляти картину так:



все стає значно простіше (хоча, з методами контейнерів типу insert( ) та erase() і є певні нюанси, про які пізніше).

- Здавалося б, проблему можна обійти так:

```
cout << "Reverse iterator base: " << *(--ritr.base()) << endl;
```

Нюанси reverse-ітераторів — 3

- І справді, якщо код скомпілюється, він даватиме очікуваний результат:

  cout << "Reverse iterator base: " << *(--ritr.base()) << endl;

  Виведе те ж, що і:

  cout << "Reverse iterator: " << *ritr << endl;

- Однак, якщо в ролі ітератора використовуються звичайні вказівники (що часто трапляється в release-варіантах реалізацій STL для vector і deque), то через правило, яке забороняє модифікувати значення, що повернули функції:

  int* fn(){**return** 0; }
  .....
  int* p = --fn(); *// Error!*

  код типу: `(--ritr.base())` не компілюватиметься. Тобто, спосіб не портабельний!
- Тоді можна зробити щось таке:

  cout << "Reverse iterator base: " << *(++ritr).base() << endl;

  Часто так — менш зручно, але завжди[6] портабельно.

[6] Звичайно, поки мова йде про сумісну із стандартом реалізацію STL.

const_iterator та iterator

- Схожа проблема існує і для const_iterator — його слід перетворити в iterator там, де потрібен виключно останній.
- Ні const_cast, ні інші приведення типів тут не допоможуть — це просто різні об'єкти!
- Якщо відомо ітератор на початок контейнера (скажімо, є доступ до контейнера), то можна зробити так: посунути відповідний константний ітератор на ту ж саму відстань:

```
ConstIterType const_itr = .....
IterType itr = .....
advance( itr , distance<ConstIterType>(itr, const_itr ));
```

  Вище IterType та ConstIterType — відповідні типи ітераторів.
- Явний вибір інстанціювання distance зроблено, тому що її аргументи матимуть різний тип, автоматичного вибору не відбудеться. А так не-константний ітератор автоматично перетвориться у константний — таке перетворення дозволене.
- Якщо ітератори НЕ є ітераторами довільного доступу (про цю класифікацію буде у лекції, присвяченій видам ітераторів), така операція потребуватиме лінійний час, тому може бути повільною. Для ітераторів довільного доступу — час константний.
- Детальніше див. книгу Скота Меєрса, "Эффективное использование STL", порада 27.

Індекси та ітератори

- Іноді в коді, що маніпулює ітераторами, треба знати номер елемента — його індекс. Знайти його можна аналогічним методом:

```
vector<int>:: iterator   itr ;
for( itr  = v.begin();  itr !=v.end();  ++itr)
{
    cout << *itr << ", index = " << distance(v.begin(),  itr ) << endl;
}
```

або:

```
vector<int>::const_iterator  itr ;
for( itr  = v.begin();  itr !=v.end();  ++itr)
{
    cout << *itr << ", index = "
     << distance<vector<int>::const_iterator>(v.begin(),  itr ) << endl;
}
```

або, в C++11:

```
for(auto  itr  = v.cbegin();  itr !=v.cend();  ++itr)
{
    cout << *itr << ", index = " << distance(v.cbegin(),  itr ) << endl;
}
```

Реалізація ітераторів

- Технічно ітератори вектора могли б бути вказівниками.
- ◇ Однак, технічно всі основні реалізації використовують спеціалізовані обгортки.
- ◇ Причина: це може зламати ряд конструкцій, наприклад:

```
void overload_resolution() {
    extern void f(std::vector<int>::iterator);
    extern void f(const char*);
    f(nullptr);                          // Проблема!
}

struct Base {};
struct Derived : Base {};

void derived_to_base_conversion() {
    std::vector<Derived>::iterator id;
    std::vector<Base>::iterator ib;
    ib = id;                             // Проблема!
}
```

- Приклади вище взято із: `https://quuxplusone.github.io/blog/2022/03/03/why-isnt-vector-iterator-just-t-star/`, там є суттєво більше подробиць.

- Повертаючись до вектора, він має конструктор, котрий приймає пару ітераторів, так званий **інтервал**, копіюючи всі значення між ними:

  vector<**int**> vcopy(v.begin()+1, v.end()−1);

  Код вище скопіює всі елементи, крім першого і останнього, із контейнера v, незалежно від типу v.

- Аналог оператора присвоєння, для копіювання частини контейнера — assign():

  vcopy.assign(v.begin()+1, v.end()−1);

  При тому, для старих елементів викликаються їх деструктори, а нові елементи створюються на їх місці, за допомогою їх конструкторів копіювання (оператор присвоєння НЕ використовується).

insert

- Метод вставки у вектор в C++98 існує в трьох варіантах:

```
// Вставити 1 елемент
iterator insert ( iterator position , const value_type& val);
// Вставити n копій
void insert ( iterator position , size_type n, const value_type& val);
// Вставити з інтервала
template <class InputIter>
void insert ( iterator position , InputIter first , InputIter last );
```

- Ітератор position вказує елемент, **перед яким** відбуватиметься вставка. Про це слід пам'ятати, особливо, коли працюється із reverse-ітераторами – їхній riter.base() переданий insert(), приведе до вставки після елемента, на який вказує riter.
- C++11 додає ще два, move-insert та insert, що вміє працювати із списками ініціалізації.
-

```
vcopy. insert ( vcopy.begin()+2, 333 );
vcopy. insert ( vcopy.end(), v.begin()+2, v.end());
vcopy. insert ( vcopy.end(), 5, 222);
int ar[]={4, 5, 6, 7};
size_t ar_size = sizeof(ar)/sizeof(ar [0]);
vcopy. insert (vcopy.begin(), ar, ar+ar_size);
```

erase

- Видалення із вектора існує в двох варіантах (C++11 лише заміняє iterator на const_iterator):

```
iterator erase ( iterator position );
iterator erase ( iterator first , iterator last );
```

Гарантії insert() та erase() щодо виключень

Insert()

- Якщо відбувається вставка одного елемента в кінець, без реалокування, гарантія строга.
- Якщо одна із цих умов не виконується (тобто, істинним є довільна комбінація із: не одного, не в кінець, відбувається реалокування), але елементи можна копіювати (або переміщати без виключень для C++11), гарантія теж строга.
- В інших випадках, гарантія базова.
- Взагалі, інші методи, типу push_back(), дають схожі гарантії — строга без реалокації, у випадку реалокації строга або базова, в залежності від обставин.

Erase()

- Реалокації не може відбуватися, тому якщо видаляється останній елемент — не кидає виключень.
- Якщо не останній — сам не кидає, однак виключення може кинути конструктор копіювання чи переміщення. В такому випадку забезпечується базова гарантія.

Інвалідація ітераторів

- Нехай є контейнер. В програмі збережено ряд ітераторів, посилань, (для вектора, також, індексів) на його елементи. Відбувається та чи інша зміна вектора — додаються або видаляються елементи, змінюється розмір, тощо. **Питання:** чи можна після цього користуватися ними?
- Відповідь: **не завжди**. Для випадків, коли ітератори (посилання, індекси) стають некоректними, вживається термін — інвалідація.
- Проблема дуже гостра. Уявимо собі код, який дублює парні елементи вектора (УВАГА: код невірний, наводиться для ілюстрації проблеми):

```
for(vector<int>::iterator itr = v.begin(); itr != v.end(); ++itr){
    if(*itr % 2 == 0){
        v.insert( itr , *itr ); ????????
        ++itr;
} }
```

Ітератор, переданий insert(), вказує на елемент, *перед* яким слід здійснити вставку. (Щоб врахувати це, і не зациклитися, пересуваємо його на наступний елемент.) Однак, чи залишився цей ітератор валідним?! А якщо це вказівник, і відбулася реалокація? І якщо інвалідуються, як продовжити обхід контейнера?

- В загальному випадку — ітератор справді не залишається валідним. Щоб не ускладнювати реалізацію, вважається, що insert() *інвалідує* всі ітератори (та посилання на елементи), від переданого йому і далі, до кінця, якщо не відбулося реалокації, і ВСІ ітератори та посилання, якщо реалокація відбулася.
- Схожі правила є і для інших операцій, що модифікують контейнери.
- Зокрема, реалокація завжди інвалідує всі ітератори.
- Повна заміна вмісту контейнера (assign, оператор присвоєння) теж інвалідує всі ітератори.
- Якщо реалокації не відбулося, push_back() інвалідує лише .end() (Тому слід акуратно пильнувати за умовами завершення циклів — хоча обчислювати v.end() на кожній ітерації може бути неефективним, але іноді це потрібно). pop_back інвалідує .end() та ітератори, що вказую на видалений елемент, erase() поводиться так само, як і insert(). І так далі, і тому подібне — див. документацію.
- Щоб була можливість продовжити, insert() повертає ітератор на перший доданий елемент. erase() повертає ітератор на нове положення того елемента, що йшов зразу після видалених.

- Тобто, приклад вище слід було писати так:

```
for(vector<int>::iterator itr = v.begin(); itr != v.end(); ++itr){
    if(*itr % 2 == 0){
        itr = v.insert( itr , *itr );
        ++itr;
    }
}
```

- Увага: під MSVC2010 цей код працює невірно – під час першої вставки підставляється якесь сміття. Так як всі решта випробуваних компілятори (gcc-4.x різних версій, на різних платформах, ICC, MSVC2013) виконують цей код нормально, не даючи жодних попереджень, та й некоректностей у ньому ніби немає, думаю, це баг в MSVC2010[7]. Наступний працює коректно і під 2010-ю студією:

```
for(vector<int>::iterator itr = v.begin(); itr != v.end(); ++itr){
    if(*itr % 2 == 0){
        int x = *itr;
        itr = v.insert( itr , x);
        ++itr;
    }
}
```

[7] Але, якщо бачите мою помилку – кажіть!

- Один із основних принципів проектування STL було розділення контейнерів і алгоритмів. По великому рахунку, вони не знають один про одного. Це дозволяє вирішити комбінаторну проблему — якщо є N контейнерів і M алгоритмів, не потрібно писати N*M функцій. Більшість алгоритмів достатньо реалізувати раз, хоча для деяких контейнерів доводиться надавати спеціалізовані версії.
- Клеєм між ними служать ітератори. **Алгоритм** працює із **інтервалами** — парами ітераторів, котрі задають, власне, інтервал даних для обробки.
- Інтервал напівзакритий:

$$[begin, end)$$

  Тобто, початковий ітератор вказує на перший елемент інтервалу, а кінцевий — на перший елемент, який НЕ входить в інтервал (може бути відсутнім, наприклад, якщо кінцевим служить щось типу v.end()) — елемент після останнього, що входить в інтервал.
- Алгоритми оголошені в файлах заголовків <algorithm> та <numeric> (з певною натяжкою і <memory>).

- Взагалі, на алгоритм можна дивитися як на певну послідовність дій, яку слід застосовувати для досягнення якогось результату. Під алгоритмами в STL маються на увазі шаблони функцій, що реалізовують такі типові послідовності дій.
- Типова така дія — сортування, реалізована алгоритмом sort:

```
vector<int> v;
  .....     заповнюємо v      .................

for(vector<int>::iterator itr = v.begin(); itr !=v.end(); ++itr)
     cout << *itr << ", ";
cout << endl;

sort(v.begin(), v.end());

for(vector<int>::iterator itr = v.begin(); itr !=v.end(); ++itr)
     cout << *itr << ", ";
cout << endl;
```

- Питання 1: Як відбувається сортування, згідно якого критерію?
- Питання 2: Що робити, якщо цей критерій не влаштовує?
- Питання 3: Чи обов'язково так розгорнуто писати банальну операцію "вивести всі елементи контейнера"?

Почнемо із першого питання. Відповідь проста:

- Сортування відбувається по зростанню.
- Для цього використовується оператор менше, "<". Тобто, sort() по замовчуванню сортуватиме будь-які типи, для яких оголошено цей оператор.

Вирішення має і проблема задання критерію сортування. Існує варіант sort(), який приймає третій аргумент — сутність, яку можна викликати як функцію. Це може бути як звичайна функція, так і об'єкт-функція:

```
bool compare(int a, int b)
{
    return a>b;
}
  .....

        sort(v.begin(), v.end(), compare);
```

Тепер сортування відбуватиметься по спаданню.

- Писати таку функцію під кожну дрібну задачку незручно.
- Узагальнення для типів робиться просто:

```
template<typename T>
bool compare(T a, T b)
{
    return a>b;
}
  .....

    sort(v.begin(), v.end(), compare<int>);
```

- Уявимо собі структуру, яку в різних ситуаціях слід сортувати згідно різного критерію:

```
struct customer_t{
    std :: string name;
    int age;
    double mass;
    long account;
};
```

Єдиний оператор порівняння не напишеш, хоча можна написати "зоопарк" функцій. Це неестетичне рішення. Що можна ще зробити? Скористатися **функторами**, див. відповідний розділ нижче.

- Питання функторів містить багато важливих аспектів, про них пізніше. Повернемося поки до алгоритмів і питання номер три

```
for(vector<int>::iterator itr = v.begin(); itr !=v.end(); ++itr)
    cout << *itr << ", ";
cout << endl;
```

Питання 3: Чи обов'язково так розгорнуто писати банальну операцію "вивести всі елементи контейнера"?

- Ні, не обов'язково. Є відповідний алгоритм, for_each():

```
template<class T>
void print_item(const T& val){
    cout << val << ", ";
}
    ..............

    for_each(v.begin(), v.end(), print_item<T>);
```

- Якщо захочеться більш гнучкої поведінки, наприклад, можливості вибирати роздільник, слід скористатися функтором – див. нижче.

Алгоритми із двома інтервалами[8]

- Їх доволі багато, наприклад, copy(), який копіює елементи одного інтервалу в інший, або transform(), який викликає до них певну функцію (функтор).
- Важливий нюанс — **інтервал, що приймає, повинен мати достатньо місця**. Автоматичного збільшення виділеного місця не відбуватиметься.
- Примітивний, неефективний спосіб використати copy():

```
vector<int> v;
vector<int> v2;
//--- Заповнення векторів ---

// Копіюємо вміст v в кінець v2
    v2.resize(v2.size() + v.size());
    copy(v.begin(), v.end(), v2.end()-v.size() );
```

- Основних недоліків у цього методу два. По перше, він неестетичний і вразливий до помилок — оці явні розрахунки нового об'єму і все таке. По друге, він повільний — під час resize() нові елементи створюються конструкторами по замовчуванню, після чого заміняються новими, в процесі копіювання.

[8] Це не є офіційним терміном, назву використано для зручності викладу.

Генератори ітераторів — 1

- reserve() в цій ситуації не допоможе (та й існує він хіба для векторів). Справа в тому, що зарезервувавши пам'ять, ще не значить збільшити кількість елементів.
- Для вирішення проблеми в файлі заголовків <iterator>, є кілька *генераторів ітераторів*: back_inserter, front_inserter, inserter, (C++11 додало ще make_move_iterator).
- Розглянемо їх на прикладі. Якщо слід один контейнер скопіювати в кінець іншого, можна зробити так:

```
copy(v.begin(), v.end(), back_inserter(v2) );
```

  Цей код дасть той же результат, що і в прикладі вище.
- Контейнер, який передається back_inserter, повинен мати метод push_back().
- Запис в елемент, на який вказує цей ітератор, автоматично додає елемент до контейнера[9], із використанням push_back(). Для цього зберігає посилання на контейнер.
- Аналогічно, front_inserter вимагає наявності push_front(), (цей метод відсутній у вектора!), і додає на початку. У всьому решта – схожий на back_inserter.

[9] Фактично, ітератор, який повертає back_inserter(), є так званим ітератором виводу. Про них пізніше.

Генератори ітераторів — 2

- Генератор inserter вимагає, щоб контейнер містив метод insert(), та при конструюванні вимагає ітератор на місце, після якого слід вставляти.
- Приклад 1:

```
copy(v3.begin(), v3.end(),
     inserter(v2, v2.begin()+3));
```

- Приклад 2:

```
copy(v3.begin(), v3.begin()+2,
     inserter(v2,
              find(v2.begin(), v2.end(), 5)));
```

Щоб приклад не був таким тривіальним, як попередні, використано ще один алгоритм — find(), який повертає ітератор на знайдений елемент, переданий йому третім аргументом, або кінець інтервалу, якщо таке значення в інтервалі не знайдено.

transform

- Приклад використання transform:

```
template <class T>
T negate_even(const T& x){
    return x%2 == 0 ? -x : x;
}

    ......................
    transform(v2.begin(), v2.end(), v2.begin(), negate_even<int> );
    // C++11:
    transform(v2.begin(), v2.end(), v2.begin(),
        []( int x) { return x%2 == 0 ? -x : x; } );
```

- Зауважте, що transform нормально працює, коли інтервал-джерело і інтервал-призначення співпадають.
- Крім "унарного" transform – за "арністю" останнього аргументу, існує і "бінарний", який бере два інтервали, застосовує до них бінарний *callable object*. Скажімо, наступний код покладе в v5 результат додавання v3 і v4:

```
template <class T>
T add(const T& x, const T& y){ return x + y; }
transform(v3.begin(), v3.end(), v4.begin(),
                back_inserter(v5), add<int> );
```

- Алгоритм find(), згаданий вище, приймає значення, яке слід знайти. Існує також алгоритм count(), який рахує, скільки раз дане значення трапилося в інтервалі:

```
cout << count(v2.begin(), v2.end(), 5) << endl;
```

- Однак, часто вони є надміру обмеженими — є потреба шукати/чи підраховувати елементи згідно певного критерію. На такий випадок існують їх варіанти, find_if() і count_if() котрі отримують замість значення так званий ***предикат*** — "callable", сутність яку можна викликати, котра повертає значення bool, і визначає, чи підходить даний елемент:

```
template <class T>
bool is_positive (const T& x){
    return x>0;
}
    .....................
  cout << count_if( v2.begin(), v2.end(), is_positive <int> ) << endl;

#if __cplusplus >= 201103L
 cout << count_if( v2.begin(), v2.end(), [](int x){ return x>0; } )
                                                  << endl;
#endif
```

Функтори

- Повернемося до прикладу вище, із друком елементів вектора. У функцію, якою користуватиметься алгоритм for_each(), неможливо передати ніяких додаткових параметрів (скажімо, стрічку-роздільник між елементами).
- Однак, алгоритму можна передавати будь-яку сутність, котру можна викликати як функцію. Наприклад, клас, котрий перевизначає оператор виклику функції, operator(). Об'єкти таких класів називають функторами або об'єктами-функціями.
- Тобто, функтори служать такими собі функціями із станом.

Функтори

```
template<class T>
struct item_printer
{
    std::string separator;
    item_printer(std::string is = ", "): separator(is){}
    void operator()(const T& val)
    {
        cout << val << separator;
    }

};
template <class T>
void print_vector( const vector<T> & v, bool print_endl = false)
{
    if(print_endl)
        for_each(v.begin(), v.end(), item_printer<T>("\n"));
    else
        for_each(v.begin(), v.end(), item_printer<T>());
    cout << endl;
}
```

Лямбда функції — C++11

- Написання тих всіх функторів для використання із алгоритмами – річ, часто, незручна. Особливо ускладнює ситуацію те, що функтори не можуть бути локальними класами[10]. Код виходить нелокальним — оголошення логіки в одному місці, використання — в іншому, створюється, по суті, багато зайвих іменованих сутностей. Звичайно, акуратно загорнуті в приватні/захищені частини класів та у простори імен, створені для реалізації, шаблони, типу розглянутих вище, спрощують задачу для користувачів, однак залишають всю складність розробникам бібліотек.
- В C++11[11] з'явилися так звані анонімні функції, або лямба-функції. Детальніше про них пізніше, а поки декілька прикладів:

```
template <class T>
void print_vector( const vector<T> & v, bool print_endl = false){
    for_each(v.begin(), v.end(), [](T val){ cout << val << ", "; });
    cout << endl;
}
```

[10] Розробники стандарту перестрахувалися, як виявилося — даремно.

[11] А ще раніше в boost.

Замикання і лямбда-функції — C++11

- Лямбда-функції вміють захоплювати змінні із охоплюючої області видимості, створюючи замикання (closure)[12]:

```
template <class T>
void print_vector( const vector<T> & v, bool print_endl = false){
  std::string sep( print_endl? "\n" : ", ");
  for_each(v.begin(), v.end(), [&sep](T val){ cout << val << sep; });
  cout << endl;
}
```

Символ & в &sep вказує захопити змінну за посиланням. Просто sep означало б – за значенням.

[12] Деталі, знову ж таки — окремо.

Приклад — предикат із станом

- В прикладі вище:

```
template <class T>
bool is_positive (const T& x){
     return x>0;
}
      .....................
   cout << count_if( v2.begin(), v2.end(),  is_positive <int> ) << endl;
```

все відносно просто. Однак, якщо іншого разу слід, наприклад, знайти числа, не більші за нуль, а більші за 7, доведеться писати новий предикат.

- Тобто, предикат із станом — річ корисна.

```
template<class T>
struct greater_than{
     T val;
     greater_than( const T& v ): val(v){}
     inline  bool operator()(const T& x ) const { return x>val; }
};
     ...................
  cout << count_if( v2.begin(), v2.end(), greater_than<int>(7) )
                                                    << endl;
```

Приклад — сортування об'єктів

- Нагадаємо собі задачу сортування згідно полів структури. Нехай є наступна структура:

```
struct customer_t{
    std::string name;
    int age;
    double mass;
    long account;
};
```

- Єдиний оператор порівняння не напишеш, великий набір спеціалізованих функцій — теж незручно. Залишається варіант із об'єктом-функцією, який пам'ятатиме, по чому слід порівнювати за кожного виклику сортування.

Почнемо із прямолінійного способу його створити:

```
class CustomerCompare1{ public:
    enum criteria_t{by_name, by_age, by_mass, by_account};
    CustomerCompare1( criteria_t ic, bool iless = true) :
        criterion (ic), use_less( iless ) {}
    bool operator()(const customer_t& a, const customer_t& b) {
        switch( criterion ) {
        case by_name: {
                int res = a.name.compare(b.name);
                if (use_less) return res<0;
                else          return res>0;
            }
            break; // Щоб не було попереджень
        case by_age:
            if (use_less) return a.age<b.age;
            else          return a.age>b.age;
            break;
        case by_mass: // ------ // ------- Аналогічно
        case by_account: // ------ // ------- Аналогічно
        default: throw std::logic_error("Unexpected sorting criterion.");
        }
    }
private:
    criteria_t criterion;
    bool use_less;
};
```

- Користуватися ним можна буде так:

```
sort(customers.begin(), customers.end(),
        CustomerCompare1(CustomerCompare1::by_age) );

sort(customers.begin(), customers.end(),
        CustomerCompare1(CustomerCompare1::by_mass) );

sort(customers.begin(), customers.end(),
        CustomerCompare1(CustomerCompare1::by_account) );

sort(customers.begin(), customers.end(),
        CustomerCompare1(CustomerCompare1::by_name) );

sort(customers.begin(), customers.end(),
        CustomerCompare1(CustomerCompare1::by_account, false) );
```

- Однак, такий підхід має два недоліки: він багатослівний, і здійснює вибір критерію під час виконання, що може значно сповільнювати програму.
- В обох випадках можуть допомогти шаблони. Почнемо із багатослівності.

- Якщо створити такий шаблон:

```
template<typename T>
bool compare(const T& a, const T& b){
    if (use_less)
        return a<b;
    else
        return a>b;
}
```

  То не треба буде кожен раз писати код, який перевіряє, "по зростанню" чи "по спаданню".
- Правда, залишається випадок стрічок — для них цей шаблон не працюватиме. Можна написати спеціалізацію, але так як спеціалізація функцій – річ сумнівна, краще — окрему функцію, яку обере при перевантаженні:

```
bool compare(const string& a, const string & b){
    int res = a.compare(b);
    if (use_less)
        return res<0;
    else
        return res>0;
}
```

- Тоді оператор виклику функції стане значно лаконічнішим:

```
bool operator()(const customer_t& a, const customer_t& b){
    switch( criterion ){
    case by_name:
        return compare(a.name, b.name);
        break; // Щоб не було попереджень
    case by_age:
        return compare(a.age, b.age);
        break;
    case by_mass:
        return compare(a.mass, b.mass);
        break;
    case by_account:
        return compare(a.account, b.account);
        break;
    default:
        throw std::logic_error("Unexpected sorting criterion.");
    }
}
```

- Однак, накладні витрати часу виконання все ще залишаться. Позбутися їх може допомогти шаблонне метапрограмування.

- Як зробити код функцій вище ефективним? Найперше — позбутися перевірки `if(use_less)`. Для цього:
- Спочатку винесемо саме порівняння в окремі функції:

```
template<typename T>
static inline bool is_less(const T& a, const T& b) { return a<b; }

static inline bool is_less(const string& a, const string& b)
    { return a.compare(b) < 0; }

template<typename T>
static inline bool is_greater(const T& a, const T& b){ return a>b; }

static inline bool is_greater(const string& a, const string& b)
    { return a.compare(b) > 0; }
```

Їм не потрібен доступ до об'єкта, тому можна зробити статичними.

- Так як часткової спеціалізації функцій не існує, два варіанти власне порівняння було винесено в окремий допоміжний клас:

І ось в цьому класі захована вся магія.

- "Загальний" випадок:

```
template<typename T, bool use_less_in>
struct compare_t {
    inline bool operator()(const T& a, const T& b) const {
        return is_less(a, b);
    }
};
```

- Та спеціалізація, яка буде вибиратися, коли use_less_in=false:

```
template<typename T>
struct compare_t<T, false> {
    inline bool operator()(const T& a, const T& b) const {
        return is_greater(a, b);
    }
};
```

- Такий собі шаблонний "if" — вибір під час компіляції, в залежності від значення use_less_in.
- Однак, клас цей незручний у використанні — слід писати щось типу: `compare_t<double, false>()(a.mass, b.mass)` — явно вказувати тип параметра, що буде явним, вибачайте за тавтологію, кроком назад.

- Щоб вирішити цю проблему, можна скористатися властивістю шаблонних функцій автоматично визначати тип (за потреби — частини!) аргументів:

```
template <bool use_less_, typename T>
inline bool compare(const T& a, const T& b ){
    return compare_t<T, use_less_>()(a, b);
}
  .............
bool operator()(const customer_t& a, const customer_t& b){
    switch( criterion ){
    case by_name:
        return compare<use_less>(a.name, b.name);
        break;  // Щоб не було попереджень
    case by_age:
        return compare<use_less>(a.age, b.age);
        break;
    case by_mass:
        return compare<use_less>(a.mass, b.mass);
        break;
    case by_account:
        return compare<use_less>(a.account, b.account);
        break;
    default:
        throw std::logic_error("Unexpected sorting  criterion .");
    }
}
```

- Використання тоді виглядатиме так:

```
sort(customers.begin(), customers.end(),
     CustomerCompare3<true>(CustomerCompare3<true>::by_age) );
sort(customers.begin(), customers.end(),
     CustomerCompare3<true>(CustomerCompare3<true>::by_mass) );
sort(customers.begin(), customers.end(),
     CustomerCompare3<true>(CustomerCompare3<true>::by_account) );
sort(customers.begin(), customers.end(),
     CustomerCompare3<true>(CustomerCompare3<true>::by_name) );
sort(customers.begin(), customers.end(),
     CustomerCompare3<false>(CustomerCompare3<false>::by_account) );
```

- Синтаксис вийшов дещо "нездоровим", але покращення може зробити код менш придатним для ілюстрації шаблонного метапрограмування, тому тут ним знехтувано. Очевидні покращенні — замість true-false використати іменовані константи, а щоб не мусити двічі повторювати <true> чи <false>, винести відповідні перерахування у базовий клас (або в охоплюючу область видимості, але тоді вони засмічуватимуть її!) або взагалі в customer_t, використати параметр по замовчуванню для порядку сортування.

- Скориставшись розглянутою методикою, позбудемося і switch часу виконання.
- Щоб синтаксис не став особливо потворним, перерахування видів порівняння винесемо в базовий клас (також, можна було його помістити зразу у вихідну структуру, customer_t):

```
class CustomerCompareProps4
{
public:
    enum criteria4_t{by_name, by_age, by_mass, by_account};
};
```

- Від якого успадкуємо клас із всіма перерахованими вище is_less(), is_greater(), compare_t, compare():

```
template<bool use_less>
class CustomerCompareBase4: public CustomerCompareProps4{
    template<typename T>
    static inline bool is_less(const T& a, const T& b){...}
                    .....................................
protected: // <==========
template <bool use_less_, typename T>
    inline bool compare(const T& a, const T& b ){...}
};
```

- Від CustomerCompareBase4 успадковуємо наш загальний шаблон, який, природно, нічого корисного робити не вміє:

```
template<CustomerCompareProps4::criteria4_t criterion, bool use_less>
class CustomerCompare4: public CustomerCompareBase4<use_less>
{
public:
    inline bool operator()(const customer_t& a, const customer_t& b) {
        throw std::logic_error("Unexpected sorting criterion.");
    }
};
```

- І створюємо його спеціалізації. Для сортування за іменем:

```
template<bool use_less>
class CustomerCompare4<CustomerCompareProps4::by_name, use_less>:
    public CustomerCompareBase4<use_less>{
public:
    inline bool operator()(const customer_t& a, const customer_t& b)
    {
        // this-> та template обовязкові'!
        // Це шаблонний метод базового класушаблону-
        return this->template compare<use_less>(a.name, b.name);
    }
};
```

- За віком:

```
template<bool use_less>
class CustomerCompare4<CustomerCompareProps4::by_age, use_less>:
    public CustomerCompareBase4<use_less> {
public:
    inline bool operator()(const customer_t& a, const customer_t& b)
    {
        return this->template compare<use_less>(a.age, b.age);
    }
};
```

- За масою та за рахунком:

```
template<bool use_less>
class CustomerCompare4<CustomerCompareProps4::by_mass, use_less>:
    public CustomerCompareBase4<use_less>
{   .....   };

template<bool use_less>
class CustomerCompare4<CustomerCompareProps4::by_account, use_less
    public CustomerCompareBase4<use_less>
{   .....   };
```

- Щоб не переписувати вручну, звичайно, варто було створити відповідний макрос для оголошення спеціалізацій

Лямба-функції, ще один приклад — C++11

- Аналогічно, з використанням лямбда-функції різко спроститься і сортування у прикладах вище:

```
sort(customers.begin(), customers.end(),
     [](const customer_t& a, const customer_t& b)
         { return a.age<b.age; });

sort(customers.begin(), customers.end(),
     [](const customer_t& a, const customer_t& b)
         { return a.mass<b.mass; });

sort(customers.begin(), customers.end(),
     [](const customer_t& a, const customer_t& b)
         { return a.account<b.account; });

sort(customers.begin(), customers.end(),
     [](const customer_t& a, const customer_t& b)
         { return a.name<b.name; });

sort(customers.begin(), customers.end(),
     [](const customer_t& a, const customer_t& b)
         { return a.account>b.account; });
```

Комбінування предикатів

- Іноді є потреба виконати операцію, дещо відмінну від тієї, для якої існує предикат. Наприклад, є розглянутий вище greater_than, а треба НЕ більше, тобто, застосувати до його результату логічне заперечення.
- Можна написати відповідну обгортку:

```
template <class Pred>
struct unary_not{
    Pred pred;
    unary_not(Pred ip): pred(ip){}

    template<class T>
     inline  bool operator()(const T& x) const{
         return  !pred(x);
    }
};
```

Скористатися нею можна так:

```
cout << count_if( v2.begin(), v2.end(),
     unary_not<greater_than<int> >(greater_than<int>(7) ) );
```

- Однак, хоча задачу воно й вирішує, але користуватися таким рішенням незручно.

- Основна незручність — необхідність вказувати ім'я типу об'єкта, для якого створюємо "заперечувач". Але шаблони функцій вміють виводити свій аргумент — цю механічну роботу можна перекласти на компілятор.

- ```
  template <class T>
  inline unary_not<T> make_unary_not(const T& pred){
      return unary_not<T>(pred);
  }
  ```

  Тоді використання стане значно простішим:

  ```
  cout <<
      count_if( v2.begin(), v2.end(),
          make_unary_not(greater_than<int>(7) ) );
  ```
  ```

Підтримка функторів стандартною бібліотекою

- Для об'єктів-функцій в <functional> оголошено базові класи, призначення яких — задати стандартні імена типам аргументів та результатів:

```
template <class Arg, class Result>
  struct unary_function {
    typedef Arg argument_type;
    typedef Result result_type;
  };

template <class Arg1, class Arg2, class Result>
  struct binary_function {
    typedef Arg1 first_argument_type;
    typedef Arg2 second_argument_type;
    typedef Result result_type;
  };
```

- Деякі *адаптери*, про які далі, потребують, щоб їх аргументи мали typedef із відповідними іменами. Найпростіше цього досягнути, успадкуваши від них.
- Вони вважаються застарілими в C++11 — рекомендується використовувати шаблон std::function.

Предикати

- Предикати — функції та функтори, що повертають bool.
- Приклад унарного предиката:

```
template <class T> struct logical_not;
```

Який оголошено якось так в C++98:

```
template <class T> struct logical_not : unary_function <T,bool> {
  bool operator() (const T& x) const {return !x;}
};
```

та так в C++11 (нагадаємо, unary_function вважається застарілим – чесно кажучи, завжди дивувався його необхідності...):

```
template <class T> struct logical_not {
  bool operator() (const T& x) const {return !x;}
  typedef T argument_type;
  typedef bool result_type;
};
```

- Наприклад, щоб замінити всі елементи певного контейнера `vector<bool> vb;` на протилежні:

```
std::transform(vb.begin(), vb.end(), vb.begin(),
                              std::logical_not<bool>());
```

- Приклад бінарного предиката:

```
template <class T> struct less : binary_function <T,T,bool> {
  bool operator() (const T& x, const T& y) const {return x<y;}
};
```

В C++11, аналогічно до logical_not, обходяться без binary_function:

```
template <class T> struct less {
  bool operator() (const T& x, const T& y) const {return x<y;}
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

- Існує і протилежний до нього:

```
template <class T> struct greater;
```

який порівнює свої аргументи із використанням оператора >.

- Таким чином, сортування по зростанню:

  sort (v.begin (), v.end());

  еквівалентне:

  sort (v.begin (), v.end (), less <**int**>);

  а по спаданню:

  sort (v.begin (), v.end (), greater <**int**>);

  і не вимагає написання власних функторів.

Список предикатів стандартної бібліотеки

Логічні операції:

- logical_and
- logical_or
- logical_not (унарний)

Порівняння:

- equal_to
- not_equal_to
- greater
- less
- greater_equal
- less_equal

Побітові логічні операції (C++11):

- bit_and
- bit_or
- bit_xor

Побітові логічні операції (C++14):

- bit_not

Арифметичні об'єкти-функції

Колекція функторів, котрі виконують стандартні математичні операції:

- plus
- minus
- multiplies
- divides
- modulus — остача від ділення
- negate — зміна знаку на протилежний, (унарний)

Скажімо, вище, щоб додати вміст двох векторів, замість писати свою функцію add(), можна було зробити так:

```
transform(v3.begin(), v3.end(), v4.begin(), back_inserter(v5),
                                plus<int>() );
```

"Заперечувачі" — negators

- Задача змінити значення предиката на протилежне теж не є надуманою, як вже було показано вище. Стандартна бібліотека включає два "заперечувачі", not1 і not2 — для бінарних і унарних предикатів, відповідно.
- Це функції, що отримують предикат, який є нащадком unary_function або binary_function відповідно (або самостійно оголошує first_argument_type і т.д.), та повертає предикат, результатом якого є логічне заперечення вихідного предикату:

```
template <class Predicate> class unary_negate
  : public unary_function <typename Predicate::argument_type,bool>{
protected:
  Predicate fn_;
public:
  explicit unary_negate (const Predicate& pred) : fn_ (pred) {}
  bool operator() (const typename Predicate::argument_type& x) const
      {return !fn_(x);}
};
template <class Predicate>
unary_negate<Predicate> not1 (const Predicate& pred){
  return unary_negate<Predicate>(pred);
}
```

not2

- Аналогічно, реалізація not2:

```
template <class Predicate> class binary_negate
  : public binary_function <typename Predicate::first_argument_type,
                            typename Predicate::second_argument_type, bool>{
protected:
  Predicate fn_;
public:
   explicit binary_negate ( const Predicate& pred ) : fn_ (pred) {}
  bool operator() (const typename Predicate::first_argument_type& x,
                   const typename Predicate::second_argument_type& y) const
  { return !fn_(x,y); }
};
template <class Predicate>
  binary_negate<Predicate> not2 (const Predicate& pred){
       return binary_negate<Predicate>(pred);
  }
```

- Таким чином, якщо згаданий вище greater_than, переписати як:

```
template<class T>
struct greater_than : public unary_function<T, bool>
{
    T val;
    greater_than( const T& v ): val(v){}
     inline  bool operator()(const T& x ) const { return x>val; }
};
```

To елементи, не більші за задане, можна порахувати так:

```
cout << count_if( v2.begin(), v2.end(),
                  std::not1(greater_than<int>(7) ) );
```

- Увага — щоб ними можна було обгортати користувацькі предикати, вони повинні мати в собі оголошені відповідні типи first_argument_type, (second_argument_type для not2), result_type, рівний bool.

- Приклад на попередньому слайді має одне негарне місце. Не зважаючи на наявність less, ми використовуємо свій greater_than, так як алгоритму потрібен унарний предикат, а less — бінарний, і байдуже, що весь час роботи алгоритму другий аргумент буде рівним 7.
- В C++98 для вирішення цієї проблеми є готові, хоч і доволі примітивні, засоби, так звані зв'язувачі (binders): bind1st, bind2nd, які із отриманого бінарного функтора роблять функтор, із одним аргументом, фіксуючи другий (перший по списку для bind1st, другий для bind2nd):

```
cout << count_if( v2.begin(), v2.end(),
                  std::bind2nd(std::greater<int>(), 7 ) );

cout << count_if( v2.begin(), v2.end(),
 std::not1( std::bind2nd(std::greater<int>(), 7 ) ) );
```

- В C++11 вони визнані застарілими, завдяки новому універсальному зв'язувачу, bind (також він доступний із бібліотеки boost):

```
cout << count_if( v2.begin(), v2.end(),
                  std::bind(std::greater<int>(), _1, 7) );
```

Де _1 вказує в ролі першого аргументу використати перший, переданий функтору, а в ролі другого – 7.

- C++98 також містить ряд **адаптерів**, призначених перетворити на функтор вказівник на функцію чи вказівник на метод: ptr_fun, mem_fun, mem_fun_ref.
- Почнемо з ptr_fun. Алгоритмам байдуже, якого виду є callable — функція, чи функтор. Але зв'язувачам чи заперечувачам — ні. Наприклад, якщо згадати функцію is_positive:

```
template <class T>
bool is_positive (const T& x){
    return x>0;
}
```

то наступний код не скомпілюється:

```
cout << count_if( v2.begin(), v2.end(), std::not1( is_positive<int>) );
```

Так як для роботи not1 не буде потрібних оголошень типів.

- Можна зробити так:

```
cout << count_if( v2.begin(), v2.end(),
                  std::not1( std::ptr_fun( is_positive<int>) ) );
```

Тобто, ptr_fun робить із унарної чи бінарної функції еквівалентний їй функтор, який є нащадком unary_function/binary_function, відповідно.

- Нехай є потреба для всіх елементів контейнера викликати певний їх метод. mem_fun повертає функтор, який викликає для свого аргументу функцію-член, задану при конструюванні посиланням на член, mem_fun_ptr — за вказівником на член.
- Наприклад, щоб заповнити вектор string_sizes розмірами стрічок із vs, можна зробити так:

```
vector<string> vs;
    ......................
vector <size_t> string_sizes (vs.size());

transform (vs.begin(), vs.end(), string_sizes.begin(),
      std::mem_fun_ref(&string::size));
```

- В C++11 ptr_fun, mem_fun та mem_fun_ref *вважаються застарілими*! Замість ptr_fun рекомендується використовувати шаблон function та ref, а замість mem_fun та mem_fun_ref — mem_fn та bind.

Адаптери C++11

- Приклади коду вище, три варіанти заміни ptr_fun:

```
cout << count_if( v2.begin(), v2.end(),
                  not1( std::function<bool(int)>( is_positive<int>) ) );
cout << count_if( v2.begin(), v2.end(),
                  not1( std::ref( is_positive<int>) ) );
cout << count_if( v2.begin(), v2.end(),
                  not1( std::cref( is_positive<int>) ) );
```

- Два варіанти mem_fun та mem_fun_ptr:

```
transform (vs.begin(), vs.end(), string_sizes.begin(),
           std::mem_fn(&string::size));

transform (vs.begin(), vs.end(), string_sizes.begin(),
           std::bind(&string::size, _1));
```

Зауважте використання bind для прив'язки об'єкта, що буде використовуватися із вказівником на член.

Адаптери в C++11 не особливо потрібні!

- Однак, C++11 надає засоби, які дозволяють, в більшості випадків, обійтися без адаптерів взагалі! Це — лямбда-функції:

```
cout << count_if( v2.begin(), v2.end(),
                  [](int x){ return !is_positive(x); });
transform (vs.begin(), vs.end(), string_sizes.begin(),
                  [](const string& s) { return s.size(); } );
```

- Код вище явно простіший, ніж всі оті збочення попередніх слайдів. В кінці кінців, й сортування за віком клієнтів, у прикладі вище, можна реалізувати так:

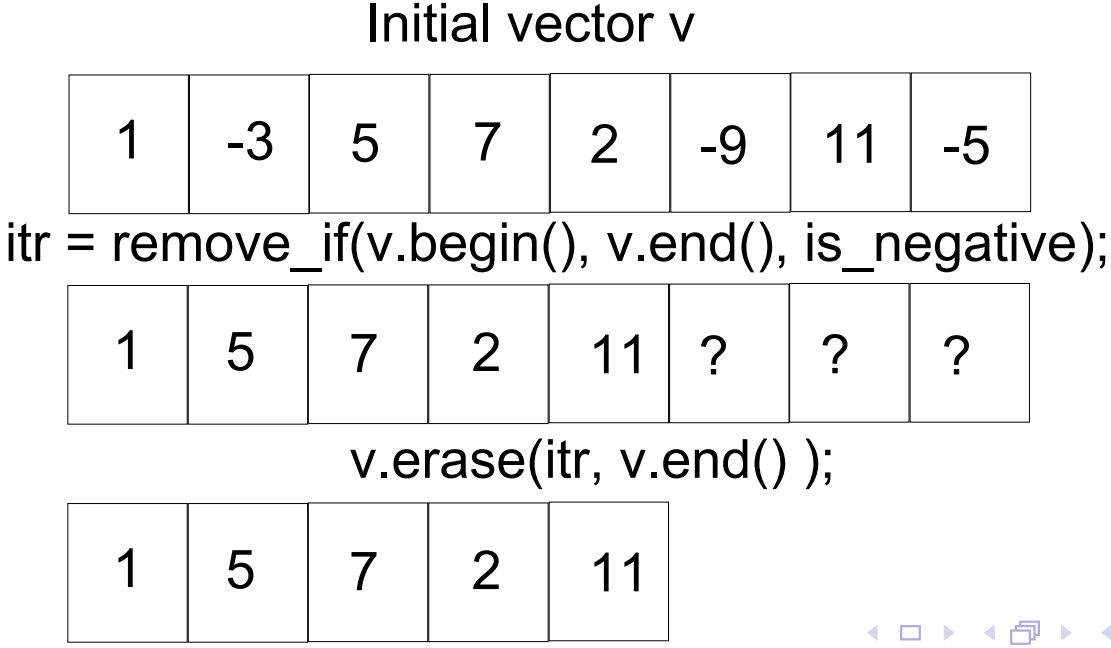
```
std::sort(std::begin(customers), std::end(customers),
    std::bind(std::less<int>(),
        std::bind(&customer_t::age, _1), // std::placeholders::_1
        std::bind(&customer_t::age, _2)));
```

Але так очевидніше:

```
sort(customers.begin(), customers.end(),
        [](const customer_t& a, const customer_t& b)
            { return a.age<b.age; } );
```

- Див. також `http://yapb-soc.blogspot.com/2015/02/common-algorithm-patterns.html`

- Алгоритм remove() видаляє елементи, рівні заданому, remove_if() — елементи, для яких предикат повертає true.
- Однак, вони, отримуючи лише інтервал, в якому слід працювати, тому **не можуть нічого видалити із контейнера**.
- Натомість, поведінка наступна — всі елементи, які залишаються, опиняються на своїх місцях. Після чого повертається ітератор на те місце, яке має стати новим end() — місце, де закінчилися елементи, що залишилися після видалення. Що буде в елементах після нього — стандартом не фіксується.
- Для зменшення розміру контейнера до потрібного, із відкиданням зайвих елементів, можна скористатися поверненим ітератором, разом із методом erase():



- ```
  vector<int> v;
      ..............
  v.erase( std::remove_if (v.begin(), v.end(),
                           std::bind2nd(std::less<int>(), 0) ), v.end() );

  v.erase( std::remove (v.begin(), v.end(), 11), v.end() );
  ```
  ```

